Actualización de Plone en genweb6

Parches de seguridad y unificación de versiones en 24 máquinas de producción

max-platform / genweb6.buildout — UPCnet
Documento de decisión: Plan A (6.0.15) vs Plan B (6.1.5)
6 de octubre de 2026

Plan: actualizar genweb6 y unificar versiones

Documento para la decisión del equipo

Contexto

Situación actual

- Producción corre Plone **6.0.6**: más de 500 webs en 24 frontends (gw6fe) y 6 backends ZEO (gw6be). Cada celda es 1 backend + 4 frontends (p. ej. gw6be1 con gw6fe1-4).
- Cada backend tiene 100 instancias. Cada frontend tiene 25 mountpoints, y su `zcdebug` solo ve esos 25.
- El advisory del 23-06-2026 trae fixes críticos: RCE por TALES (severidad 9.9), DoS por importación de iCalendar (9.1), DoS por el portlet RSS (9.1) y DoS por longitud de título/descripción/filename (6.5). Además está el advisory del 05-06-2026 sobre `plone.app.textfield` y `plone.restapi`.
- Cada máquina tiene paquetes distintos según cuándo se instaló, porque muchos paquetes no están pineados en `versions.cfg`.

Objetivo

1. Aplicar los parches de seguridad.
2. Llegar a la versión más reciente posible sin reescribir los paquetes Genweb.
3. Tener un `versions.cfg` con todos los paquetes pineados, generado en local a partir de la instancia que funciona. Así, con `$venv/bin/buildout -N -c zope-only.cfg`, todas las máquinas bajan y quedan en las mismas versiones.

Datos que condicionan el plan

- Últimas versiones publicadas: **6.0.15** (marzo de 2025, la 6.0 está en modo solo seguridad) y **6.1.5** (25-06-2026, ya incluye los fixes). El soporte de seguridad de **6.0 y 6.1 termina el mismo día, 31-12-2027**. La 6.2 tiene soporte hasta Plone 7, previsto para mediados de 2028.
- Ya existe la rama `plone_6_0_15` en genweb6.buildout (septiembre de 2025, sin mergear), con el bump de `requirements.txt` (zc.buildout 4.1.4, setuptools 75.8.2, pip 25.0.1). **El Plan A se hace sobre esta rama ya existente.**
- La URL `dist.plone.org/release/6.0.6/versions.cfg` aparece en 3 sitios: `genwebupc.cfg`, `zope-only.cfg` y `zeo-only.cfg`.
- `collective.exportimport` y `contentimport` se quedan como están, desplegados desde `src/` también en producción. `collective.newrelic` queda fuera de este plan: solo está en evaluación en local.
- Ningún paquete Genweb limita la versión de Plone. El riesgo está en el código propio: 22 overrides de plantillas (jbot) en `genweb6.core`, unos 45 monkeypatches, y el parche de TinyMCE.
- **Ansible se usa tal cual está**: `genweb_actuacio` hace git pull + bootstrap + reinicio de los frontends; `genweb_upgrade_plone` ejecuta el upgrade en cada frontend sobre sus 25 mountpoints, cubriendo así los 100 de cada celda.
- **Backups:** antes de empezar, Ops hace snapshot de las máquinas. La actuación se hace en sábado; como último recurso queda el backup del viernes.

> **¿Hay que actualizar el ZEO?** No es necesario para aplicar los parches: el servidor ZEO solo ejecuta ZODB/ZEO y no carga código de Plone; todos los fixes del advisory están en paquetes que cargan los clientes Zope (frontends). En PRE hay que comprobar que los clientes con el ZODB/ZEO nuevo conectan bien al servidor actual. El backend se puede reconstruir más adelante, fuera de la ventana crítica.

Base común: generar el `versions.cfg` completo en local

1. En el buildout local, con la versión de Plone elegida, ejecutar `bin/buildout -N -c genwebupc.cfg` y comprobar que `./bin/instance fg` arranca y funciona con datos reales.
2. Sacar las versiones de todos los eggs que carga `bin/instance` (rutas `eggs/<nombre>-<versión>` del `sys.path`), o usar la salida de `show-picked-versions = true` durante ese buildout.

3. Volcarlas a `versions.cfg`, en este orden: pins UPCnet, hotfixes de seguridad (cada uno con su advisory), y el resto de paquetes resueltos en local.
4. Excluir los develop eggs de `src/`: `collective.exportimport` y `contentimport`.
5. Limpieza de duplicados (`prompt-toolkit`, `genweb6.patents`) y actualización de la URL de extends en los 3 cfg. Crear un tag en el repo para la versión desplegada.

Despliegue (los dos planes)

PRE (copia real de un backend) → celda canaria un sábado → resto de celdas en los sábados siguientes. En cada celda:

1. Ops hace el snapshot de los 4 frontends y del backend.
2. Merge de la rama a `master`.
3. `genweb_actuacio` en los 4 frontends.
4. `playbook-check-upgrade-plone.yml` (dry run).
5. `playbook-upgrade-plone.yml`.
6. `bulk_action` de Genweb, si hace falta.
7. Smoke tests.

Plan A (recomendado): 6.0.15 + hotfixes ahora

> **Por qué:** la mínima superficie de cambio para 500 webs, los paquetes funcionan casi sin tocar y el parche se puede desplegar pronto.

1. **Rama** `plone_6_0_15` (la que ya existe), actualizada con `master`:
   - Usar los pins genweb6 de `master` (core 1.92, theme 1.58, upc 1.36…) en lugar de los antiguos de la rama (core 1.71…).
   - Quitar `branch=plone_6_0_15` de core y theme en `sources.cfg`, salvo que esas ramas tengan cambios necesarios.
   - Revisar los pins que trae la rama (`z3c.jbot`, `plone.app.mosaic`, `plone.app.multilingual`, `plone.app.z3cform`, `pyOpenSSL`, `cryptography`).
2. **URL de extends:** `dist.plone.org/release/6.0.15/versions.cfg`.
3. **Hotfixes en** `versions.cfg`, sacados del advisory:

   ```
   plone.app.contenttypes = 3.0.12
   plone.app.dexterity    = 3.2.3
   plone.app.event        = 5.2.4
   plone.app.portlets     = 5.0.8
   RestrictedPython       = 8.3
   plone.app.textfield    = 2.0.2
   plone.restapi          = 9.15.6
   icalendar              = <versión corregida de la línea que usa 6.0.15>
   ```

   Además, revisar los anuncios de plone.org/security entre marzo de 2025 y hoy por si falta algún otro hotfix.
4. **Pins UPCnet a revalidar:** `plone.session=4.0.3` (SSO, probar CAS antes de quitarlo), `plone.api=2.3.0`, `Products.PloneLDAP=2.0.0a1`, `eea.facetednavigation=16.0a1`, `collective.easyform=4.1.4`, `collective.z3cform.datagridfield=3.0.0`.
5. **Revisión de seguridad del código propio (crítico):** comparar con diff los overrides jbot y los monkeypatches que tocan código afectado por los fixes, porque pueden **reintroducir la vulnerabilidad**:
   - `plone.app.portlets.portlets.rss` y el `Renderer` del portlet de calendario.
   - `plone.app.event.browser.event_listing` y el patch de `EventListing`.
   - Los listados de `plone.app.contenttypes`, más los patches de `CollectionView` y `FolderView`.
   - `Products.CMFPlone.browser.syndication.templates.RSS`.

   Aplicar los cambios del upstream fijado y publicar nuevas versiones de genweb6.* si hace falta.
6. Generar el `versions.cfg` completo (base común) con esta versión.
7. **Despliegue:** reinicio escalonado dentro de la celda (6.0.6→6.0.15 no cambia los pickles); lanzar el upgrade cuando los 4 frontends ya estén en la versión nueva; el ZEO no se toca.

Fase 2 (más adelante): como 6.0 y 6.1 tienen el mismo fin de soporte, el salto que de verdad alarga la vida útil es a **6.2.x Classic**. El trabajo de código es el mismo que en el Plan B.

Plan B: directo a 6.1.5

> **Por qué:** una sola migración. Requiere trabajo de adaptación focalizado (no una reescritura) y retrasa el parche de seguridad varias semanas. **Si se elige, lo razonable es aplicar antes el Plan A (pasos 1 a 5) como parche urgente.**

1. **Ramas** `plone61` en el buildout y en cada paquete afectado. Publicar versiones mayores (p. ej. genweb6.core 2.x) para que la línea 1.x siga recibiendo parches en 6.0.
2. **URL de extends:** `dist.plone.org/release/6.1.5/versions.cfg`. Ya trae contenttypes 4.0.10, dexterity 4.1.3, event 5.2.4, portlets 6.0.4 y RestrictedPython 8.3. Comprobar `plone.app.textfield ≥ 3.0.2` y `plone.restapi ≥ 9.15.6`.

3. **Trabajo de código:**
    - 22 overrides jbot: diff contra los originales de 6.1 (más riesgo: `search`, `login`, `listing*`, macros/widget de z3cform, `column`; en serveistic, 5 plantillas de faceted).
    - Reescribir el parche de TinyMCE (6.1 trae TinyMCE 7 y cambia el mockup).
    - Revisar uno a uno los ~45 monkeypatches.
    - Quitar dependencias legacy (`plone.app.form`, `plone.formwidget.contenttree`) y migrar los portlets `zope.formlib` de serveistic.
    - Imports a actualizar: `RelatedItemsFieldWidget`, `safe_unicode` → `plone.base.utils.safe_text`.
    - Validar subclases: portlet de login, renderers de news/navigation, viewlets de title y dublincore.
    - Add-ons compatibles con 6.1: mosaic, easyform, datagridfield, faceted, PloneLDAP, createdx, ftw.casauth, exportimport.
    - Ejecutar `./bin/test -s <paquete>` en todos los paquetes.
4. **Python:** 3.11 sirve (6.1 soporta de 3.10 a 3.13).
5. Generar el `versions.cfg` completo (base común) con 6.1.5.
6. **Migración de datos:** upgrade steps 6.0→6.1 en los 500 sites. Medir el tiempo por site en PRE con un backend completo.
7. **Despliegue:** ventana completa por celda (parar los 4 frontends, actualizar, upgrade, arrancar); no mezclar frontends 6.0 y 6.1 contra el mismo ZEO; el backend se puede actualizar después.
8. **Nota:** con el mismo esfuerzo se podría apuntar directamente a **6.2.x**, que tiene más vida útil.

Estimación de horas

Supuestos: 1 persona que conoce el código de Genweb. Rangos orientativos (mín–máx); no incluyen el tiempo de Ops para los snapshots.

Tarea	Plan A (6.0.15)	Plan B (6.1.5)
Rama del buildout, requirements.txt, extends	4–6 h	6–8 h
Buildout local, arrancar la instancia y generar el versions.cfg completo	8–12 h	10–14 h
Hotfixes y revalidar pins UPCnet (SSO/CAS, LDAP, faceted, easyform)	8–12 h	incluido en add-ons
Revisión de seguridad de overrides y patches afectados, releases genweb6.*	12–16 h	incluido abajo
Diff y adaptación de los 22 overrides jbot (+5 de serveistic)	–	30–45 h
Reescribir el parche de TinyMCE 7	–	16–24 h
Revisar unos 45 monkeypatches	–	30–40 h
Dependencias legacy (formlib, form, contenttree), imports, subclases	–	20–30 h
Add-ons compatibles (mosaic, easyform, datagrid, faceted, LDAP, casauth…)	–	24–40 h
Ramas y releases de ~11 paquetes	2–4 h	8–12 h
Tests unitarios y corrección	4–8 h	24–40 h
Upgrade sobre copia real de un backend y medición del tiempo	6–8 h	12–16 h
PRE: despliegue, smoke tests funcionales y correcciones	12–16 h	40–60 h
Producción: 6 celdas en sábados (≈2–3 h/celda en A, ≈4–6 h/celda en B)	12–18 h	24–36 h
Documentación y coordinación con Ops y el equipo	4–6 h	8–12 h
Total	**≈ 70–105 h (2–3 semanas)**	**≈ 270–380 h (7–10 semanas)**
Hotfix previo urgente (Plan A, pasos 1–5) si se elige B	–	+35–50 h

Lectura:

- El Plan A cierra las vulnerabilidades críticas en unas 2–3 semanas y unifica versiones.
- El Plan B cuesta unas 4 veces más y lleva al mismo fin de soporte de seguridad (31-12-2027) que el Plan A.
- Si se decide invertir en el salto de versión, conviene que sea directamente a 6.2, con un coste similar al del Plan B.

Ficheros clave

- **genweb6.buildout:** rama `plone_6_0_15`; `genwebupc.cfg`, `zope-only.cfg`, `zeo-only.cfg`; `versions.cfg`, `sources.cfg`, `requirements.txt`.
- `src/genweb6.core`: `templates/` (jbot), `patches.py` y `patches.zcml`, `setup.py`.
- **genweb.ansible:** `genweb_actuacio/` y `genweb_upgrade_plone/`, sin cambios.

Verificación

1. **Local:** el buildout termina sin errores y `./bin/instance fg` funciona; `./bin/test -s <paquete>` en cada paquete; `@@plone-upgrade` sobre una copia real de Data.fs; un buildout limpio en otro directorio da exactamente los mismos eggs.
2. **Seguridad:** comprobar en `bin/zc1` las versiones de los paquetes del advisory; probar a mano el portlet RSS con un feed enorme, un .ics malformado y un título de varios MB.

3. **PRE:** upgrade (dry run y real) en los 4 frontends de una celda; smoke tests (login CAS/LDAP, búsqueda, listados, eventos, TinyMCE, easyform, faceted, multilingual, exportimport); conexión de los clientes nuevos al ZEO actual.
4. **Producción, en cada celda:** `/ping` de cada zc y URL públicas de cada mountpoint a través de varnish y nginx; comparar el `sys.path` de `bin/zc1` en los 4 frontends; revisar New Relic antes de pasar a la celda siguiente.